



SDEV2171

Lesson 07: Handling User Input

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain how user input is captured in React Native applications
- Use common input components to collect user data
- Handle basic input-related events
- Trigger simple screen-level behaviour in response to user input and user actions
- Debug common issues related to user input handling

Lesson Roadmap

1. Reconnect lesson-06 navigation to user interaction
2. Introduce `TextInput` , `Pressable` , and event handling
3. Analyze a simple input-response flow
4. Guided interaction build
5. Testing and troubleshooting
6. Checkpoint and lesson-08 handoff

Starting Point From Lesson 06

- use the same Expo Router project from lesson 06
- confirm the app still opens on the home screen
- confirm the second screen still opens and returns correctly

If your project is not ready

- if your lesson-06 app is `configured` , keep using your own project
- if your project is `partial` or `blocked` , follow your recorded remediation steps first
- if you cannot recover in time for class progress, use the instructor-guided reference path and record your next step

Why Input Matters

- navigation lets users move between screens
- input lets users affect what happens on those screens
- even a simple text field or button changes the app from static to interactive
- later lessons build on these interaction patterns

Two Core Components For Today

Component	Job
TextInput	captures text from the user
Pressable	detects a press interaction

Two core events

- `onChangeText` responds when the text changes
- `onPress` responds when the user activates a button or press target

Input Flow For This Lesson

1. the user types into `TextInput`
2. the app reads the new text
3. the user presses a button
4. the screen shows a visible response

Important framing

- the lesson is about input and response
- any changing-value code is an implementation detail that supports the lesson

Plain-language memory rule

- for today, the screen remembers two temporary values:
 - the typed text and the response text shown after the action

Code name for that helper

- the example code uses `useState` for those two temporary values
- for this lesson, treat `useState` as a small screen-memory helper, not as the main theory target

Rough Interaction Versus Clear Interaction

Weak interaction

- the user can type, but nothing visibly changes
- the button exists, but its result is unclear
- the screen gives no feedback

Clear interaction

- the user knows where to type
- the press action has a clear purpose
- the screen shows what changed after the action

Minimum Input Pattern

```
import { useState } from 'react';

const [note, setNote] = useState('');

<TextInput
  value={note}
  onChangeText={setNote}
  placeholder="Type a short note"
/>
```

What this means

- `note` is the current text value
- the field shows a current value
- `onChangeText` receives the updated text
- the screen can use that text in later behaviour
- `useState` is the helper that lets the screen remember the text

Minimum Press Pattern

```
const [responseText, setResponseText] = useState('No preview yet.');
```

```
function handlePreview() {  
  setResponseText(`Preview: ${note}`);  
}
```

```
<Pressable onPress={handlePreview}>  
  <Text>Preview response</Text>  
</Pressable>
```

What this means

- the user activates a clear action
- the screen runs one handler when pressed
- the result should be visible and easy to test
- `responseText` is the second temporary value the screen remembers

Guided Build Order

1. confirm the lesson-06 navigation baseline
2. edit `src/app/details.js`
3. add a `TextInput` to the detail screen
4. add one `Pressable` action
5. create one visible feedback area
6. test typing and pressing
7. keep the clearest working interaction

Visible Behaviour Target

- the home screen still navigates to the detail screen
- the detail screen accepts text input
- one button triggers a visible response
- the response changes based on the user action
- the screen still has a clear return path

Event Mental Model

- `onChangeText` runs when the text changes
- `onPress` runs when the user activates the press target
- a visible response should make it obvious that the event happened

Plain-language version

- typing should do something understandable
- pressing should do something visible

Recovery Path If Interaction Is Blocked

- `configured` : input works, the press action works, and the screen response is visible
- `partial` : the input or press flow exists, but one part still fails
- `blocked` : the interaction flow cannot be completed during class

If you are blocked

- use the instructor-guided reference path
- compare your `src/app/details.js` to `example/input-handling-reference/src/app/details.js`
- repair one part at a time in this order:
 - the `useState` import and temporary values
 - the `TextInput`
 - the `Pressable`
 - the visible response area
- record the failing step and one next action before class ends

Common Input Mistakes

- `TextInput` appears, but the value never changes visibly
- the press target exists, but `onPress` is missing
- the button runs, but the screen gives no clear feedback
- the user types on the wrong screen
- too many interaction changes happen before retesting

Quick Interaction Reasoning Example

If this event exists

- `onChangeText`

Then you should be able to explain

- what text changed
- where that text is used next

Debug rule

- check the input field first
- then check the press handler
- then check the visible response area

Guided Experimentation

- change the placeholder text
- change the button label to make the action clearer
- try one empty input and one non-empty input
- test the response twice in a row
- keep the clearest working version

Interaction Status Check

Before the checkpoint, record one state

- configured
- partial
- blocked

Every student should also record

- the current failing step, if any
- the next action to try before lesson 08

Checkpoint

Students should be ready to show

- a screen with working input and a visible response

Students should also be able to answer

- Which component captures text?
- Which event responds to text changes?
- Which event responds to the button press?
- What visible behaviour changed after the action?

If the interaction is still partial or blocked

- record the failing step
- record the next remediation action
- mark the lesson target as not yet mastered

What We Learned Today

- how `TextInput` captures text
- how `Pressable` detects user actions
- how `onChangeText` and `onPress` connect interaction to behaviour
- how to test whether an interaction is really working
- how to classify progress as configured, partial, or blocked

Next Lesson Bridge

- keep this same interaction project
- lesson 08 consolidates styling, navigation, and interaction skills
- next class uses this app as the base for guided exercises and applied practice

Ready for lesson 08 means

- the app still runs
- the navigation flow still works
- the input and visible response flow is configured
- if your status is partial or blocked , you need remediation before lesson 08